

An Implementation of Open-RMF in an office environment

Introduction

ROS 1 has reached the EOL

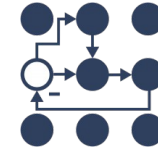
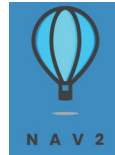


Better to move to ROS 2?



With ROS 2:

- Old packages are rewritten
- New packages are born



At Elettra Sincrotrone IT Group, we want to see the Open-RMF capacities and upgrade our AMRs, so we:

- Build 2 AMRs with ROS 2 creating 2 fleets:
 - One from an UGV Mini
 - One porting a Jobot AMR from ROS 1 to ROS 2
- Port the DStar global planner from ROS 1 to ROS 2 to be used in our robots
- And, finally, use Open-RMF as middleware for these two fleets

ROS Components

- Middleware

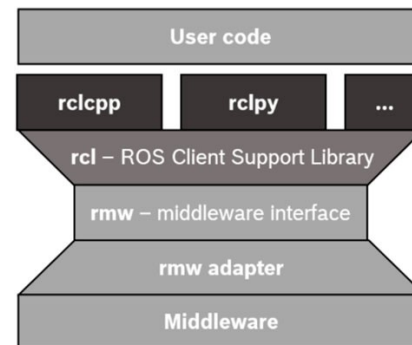
- DDS (eProsima Fast, Eclipse Cyclone, ...)
- No DDS (Eclipse Zenoh)

- Code:

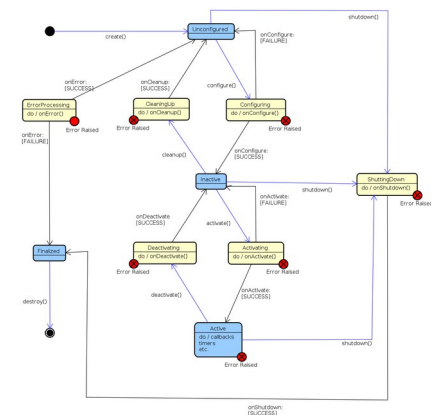
- Multiple client libraries : rcl, rclcpp, rclpy, rclrs, ...
- Topics, Service, Action
- Lifecycle
- Composition

- Tools:

- rqt, launch, rviz, gazebo, ...



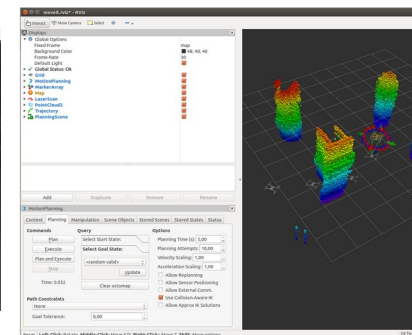
Structure



Lifecycle



Gazebo

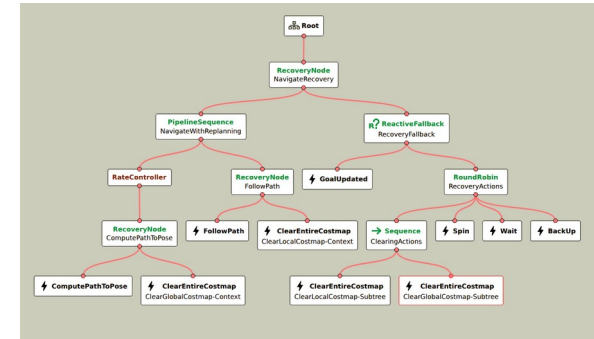
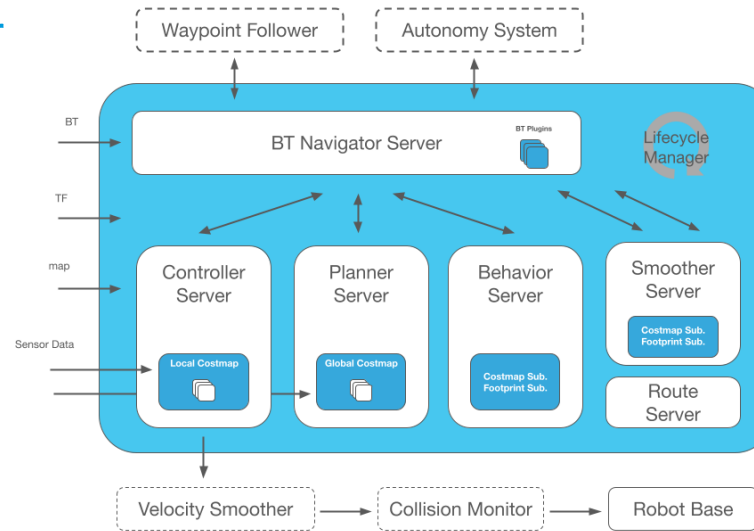


Rviz

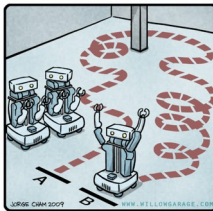
Nav2 project

- Structure

- BT Navigation Server
- Behavior Server
- Planner Server
- Controller Server
- Smoother Server
- Route Server
- Lifecycle manager



R.O.B.O.T. Comics



Nav2 Requirements

- Need a TF tree based on REP 105
 - base_link → odom → map
 - base_link → odom (continuous link)
 - Given by robot estimation (wheel encoders, ...)
 - ROS packages (robot_localization, fuse, ...)
 - Odom → map (can be a discrete link)
 - SLAM (Cartographer, ...)
 - AMCL
 -

Build the AMRs

Mini:

Unmanned Ground
Vehicle



Control the movement and give
information about its state

Sensors

IMU



LIDAR



Give more information
about the system or about
the world

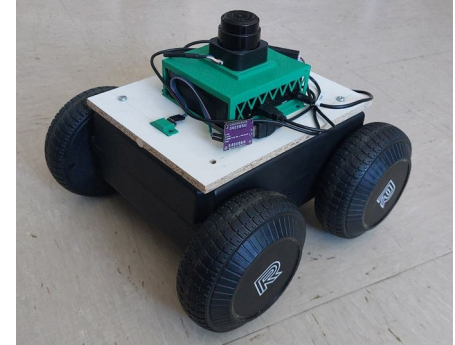
PC



Take all the data and
control the robot without
human interaction



AMR



Jobot:



- ROS 1 AMR that get the data from the encoders and a IMU using a serial communication with an Arduino
- It has also a LIDAR for mapping



Docker with



Upgrade Jobot AMR - ROS 1 → ROS 2

- Build system: catkin → colcon (ament_cmake)
- rclcpp with more C++ 17 features, like smart pointer and new cmake
- Naming
- ROS msgs update
- Lifecycle implemented
 - In on_configure() : add creation of secondary nodes (IMU), setup parameters and create callbacks
 - In on_deactivate() : cleaning
- Find ros-serial for ROS 2

Updated DStar global planner and results

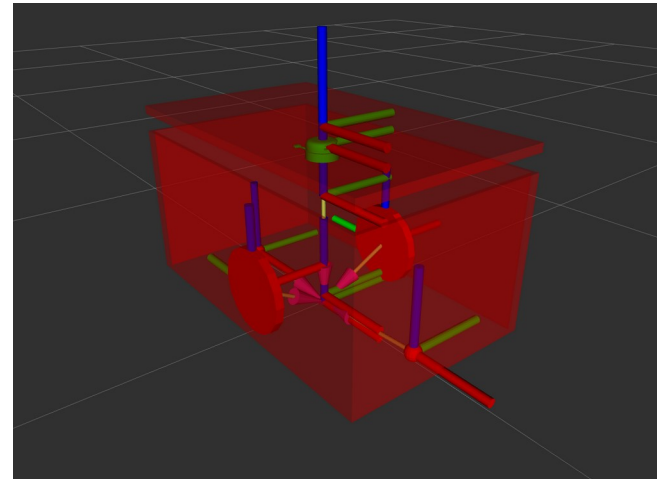
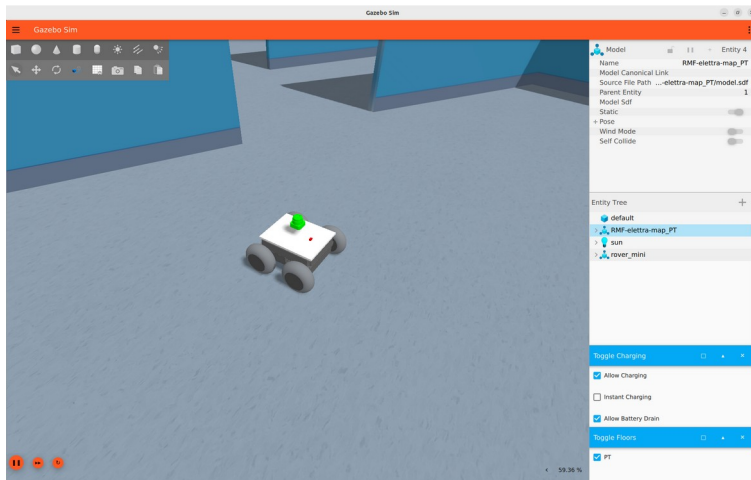
- Upgrade planner
 - Inheritance from `nav2_core::GlobalPlanner`
 - Lifecycle structure
 - `configure()` : setup parameters and get pointer
 - `activate()` : fill DStar costmap
 - `cleanup()` : destroy allocate memory
 - `createPlan()` : return a path
 - plugin
- Results:



Start->End	ROS 2 D-start (s)	ROS 2 NavfnPlanner (s)
office -> storage room	1.00	0.020
office -> entrance	5.47	0.150
entrance -> CAD designers	5.36	0.130

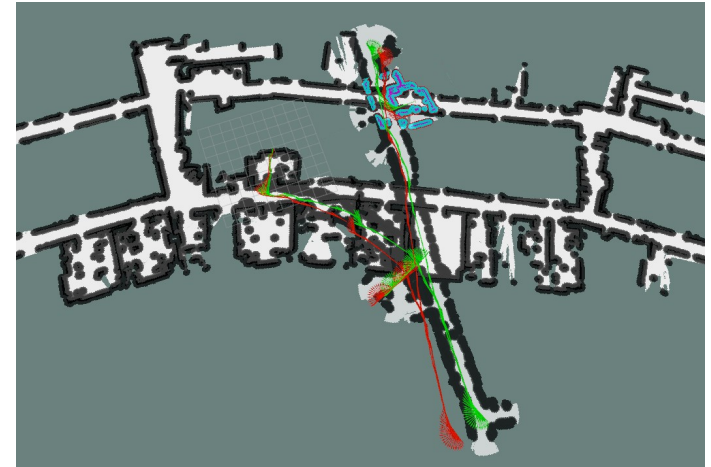
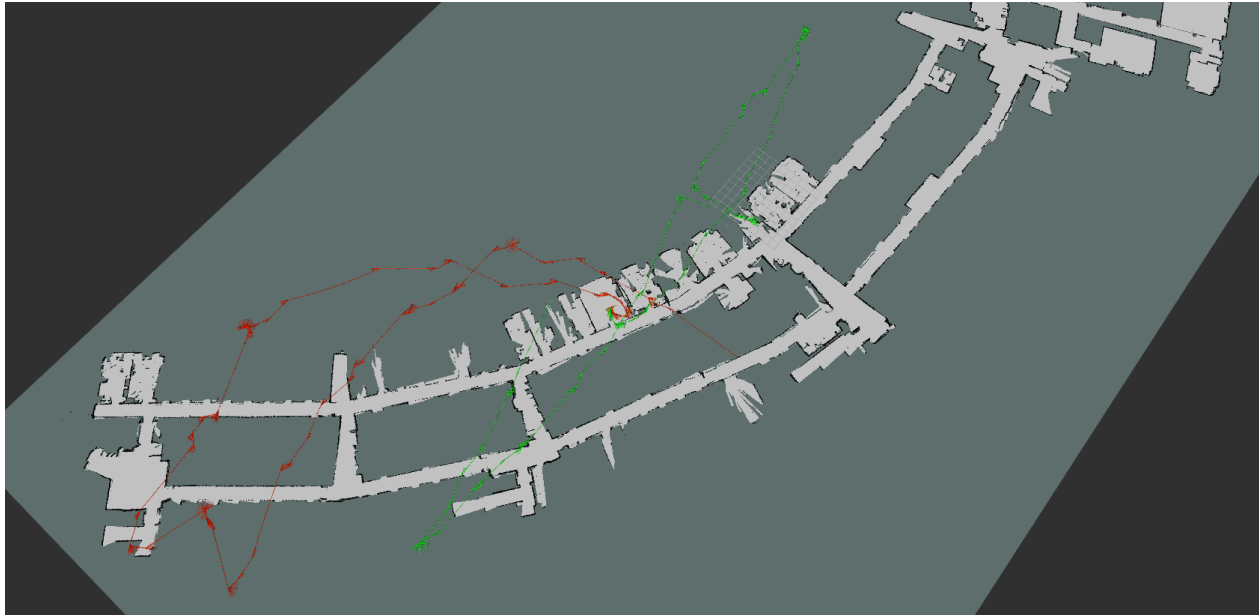
Adding also simulation

- Mini: Edit URDF for sensors position
- Jobot: Create an URDF description with sensors and movement control (DiffDrive)



UGV - Problems

- Their returned odometry → need an improvement

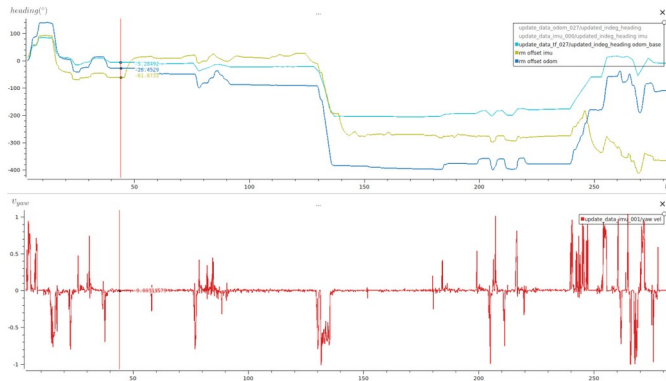


How to improve Odometry

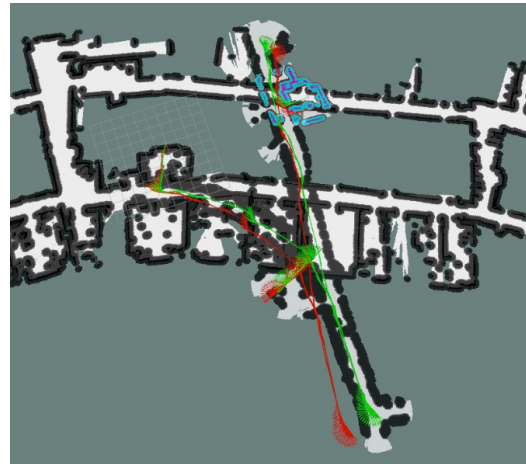
- Using packages that fuse sensors data:
 - robot_localization that fuse IMU data with odometry given by robot
- And tuning that packages
 - tuning the parameters of the EKF
- Moving the robot and recording the data using rosbag2
 - Repeatability
 - Speed

How to tune EKF of robot_localization

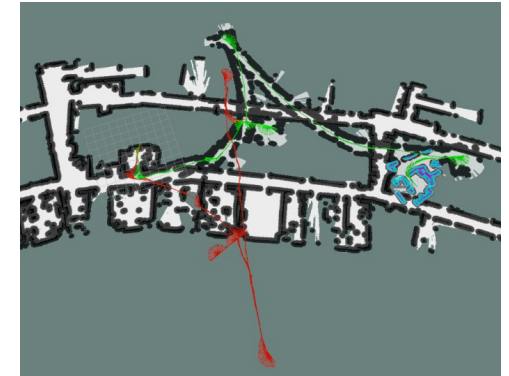
- Choose what sensors use and how



Mini error in heading



Jobot with yaw from odometry



Jobot with v_{yaw} from odometry



Jobot with v_{yaw} from odometry and IMU

How to tune EKF of robot_localization

- Change the process noise covariance matrix Q

$$x_k = f(x_{k-1}) + w_{k-1}$$

$$z_k = h(x_k) + v_k$$

Prediction step

$$\hat{x}_k = f(x_{k-1})$$

$$\hat{P}_k = F P_{k-1} F^T + Q$$

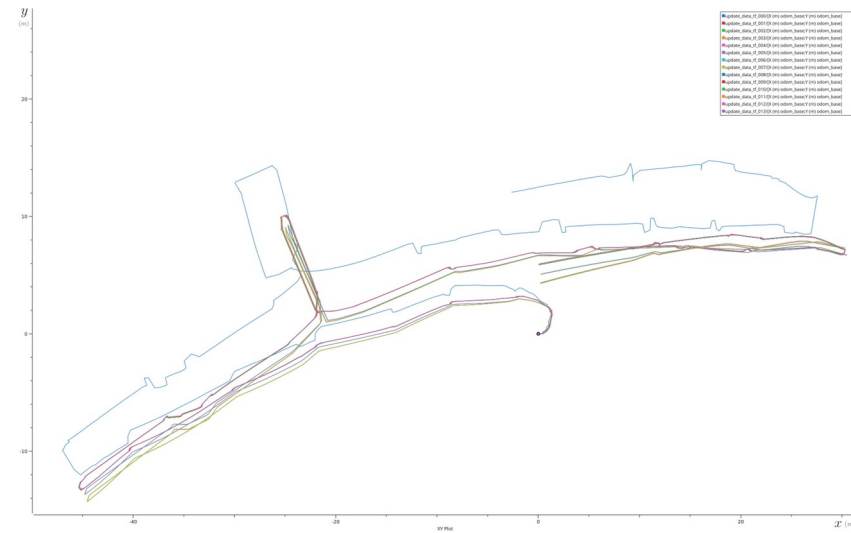
Correction step

$$K = \hat{P}_k H^T (H \hat{P}_k H^T + R)^{-1}$$

$$x_k = \hat{x}_k + K(z - H \hat{x}_k)$$

$$P_k = (I - KH) \hat{P}_k (I - KH)^T + K R K^T$$

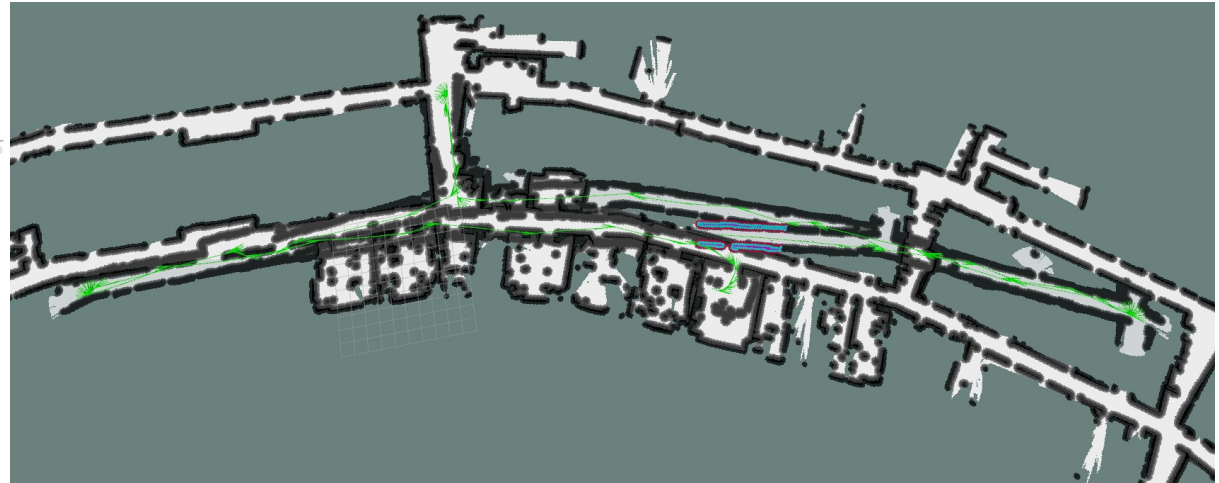
Results Mini



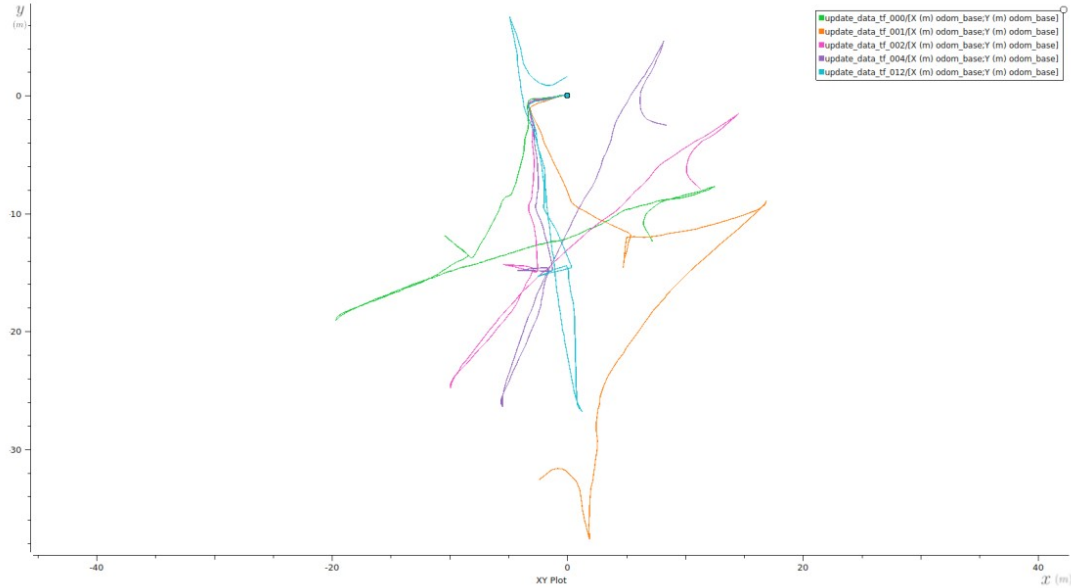
v_x (-)	v_y (-)	v_{yaw} (-)	data name	diff start end (m)
0.001	0.001	0.001	000	12.36
0.01	0.01	0.01	001	5.88
0.1	0.1	0.1	002	5.10
10.0	10.0	10.0	003	4.37
50.0	50.0	50.0	004	4.31
0.001	0.001	0.1	005	4.83
0.001	0.001	10.0	006	4.33
0.001	0.001	50.0	007	4.27
0.001	0.1	0.001	008	5.88
0.001	10.0	0.001	009	5.88
0.001	50.0	0.001	010	5.88
0.1	0.001	0.001	011	5.94
10.0	0.001	0.001	012	5.94
50.0	0.001	0.001	013	5.97

NB:

Still translation error
Still rotation error



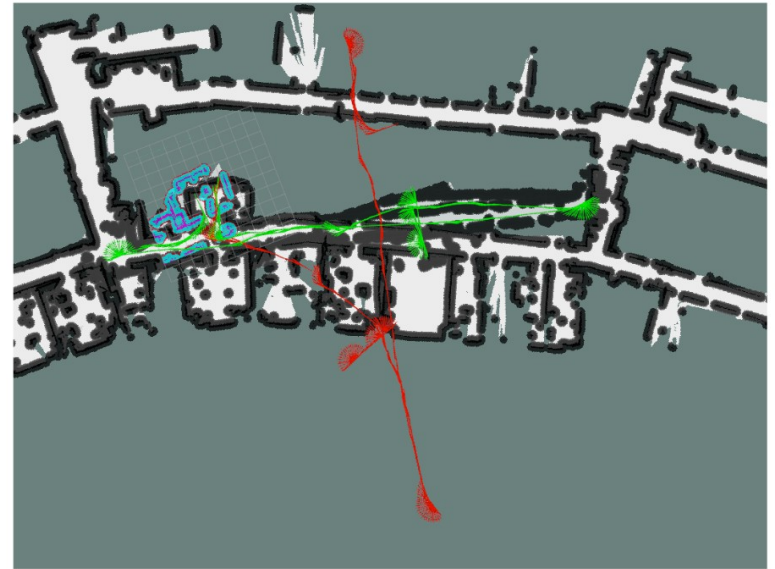
Results Jobot



NB:

Little translation error
Still rotation error

v_{yaw} (-)	record name	diff start end (m)
-	000	14.28
-	001	32.67
0.1	002	13.88
10.0	003	13.26
100.0	004	8.74
200.0	007	4.31
300.0	008	1.83
400.0	009	3.50
320.0	012	1.63



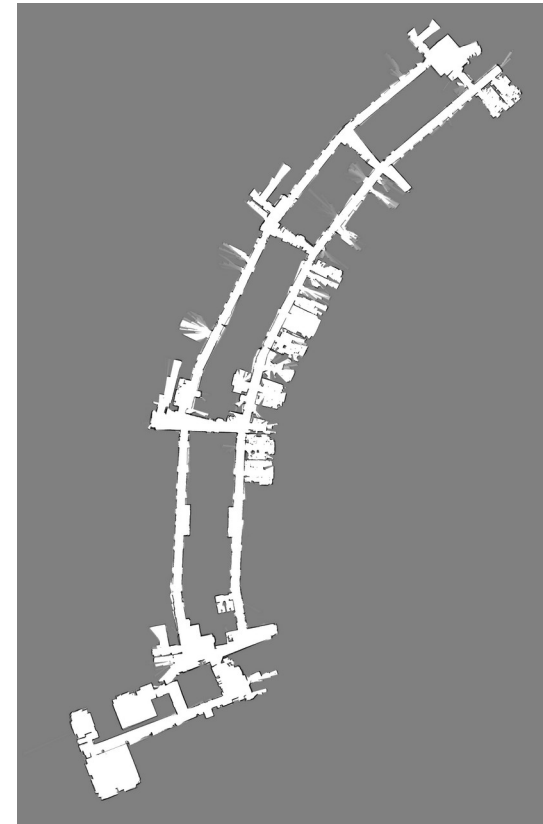
Global localization

We improve but still errors

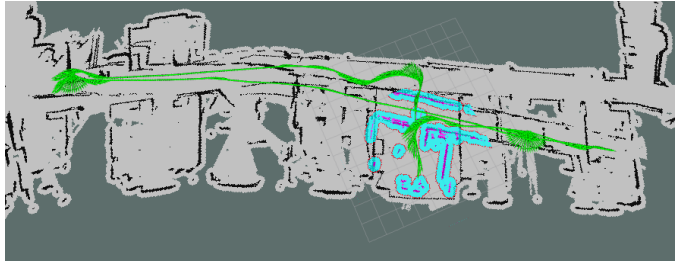


correction using a map

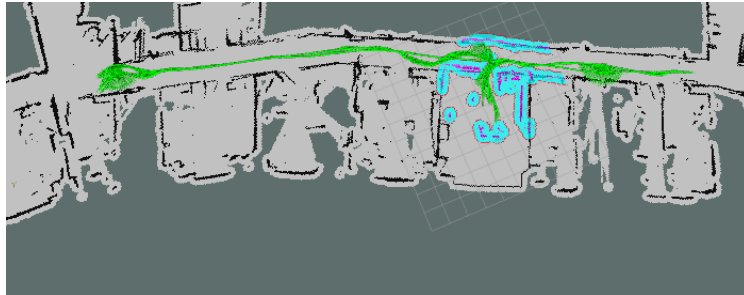
- SLAM or only localization?
 - Localization:
 - Cartographer
 - AMCL
 - Tuning:
 - α parameters :
Expected process noise in odometry's
rotation/translation estimate from
rotation/translation



AMCL calibration Mini



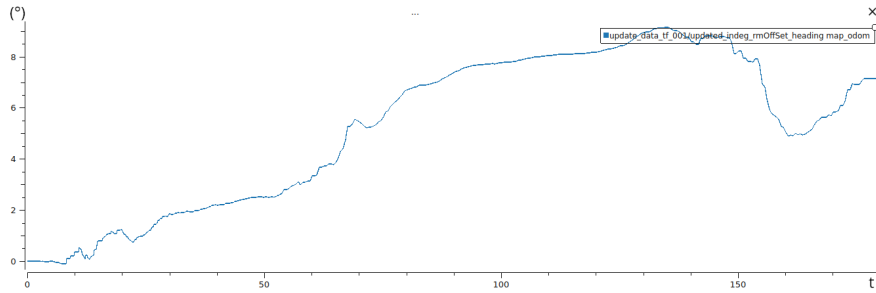
Correcting rotation



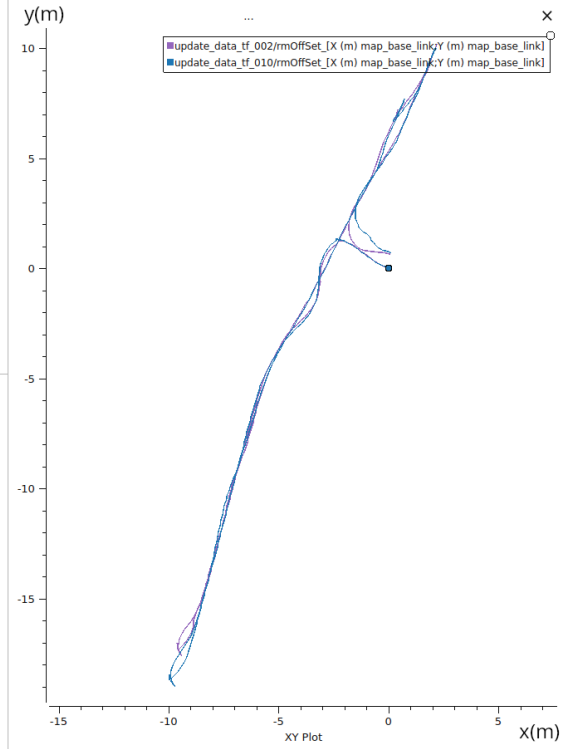
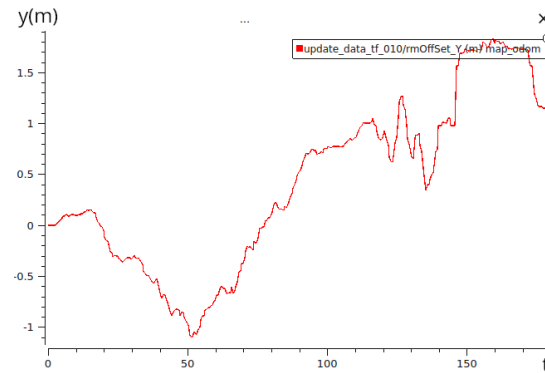
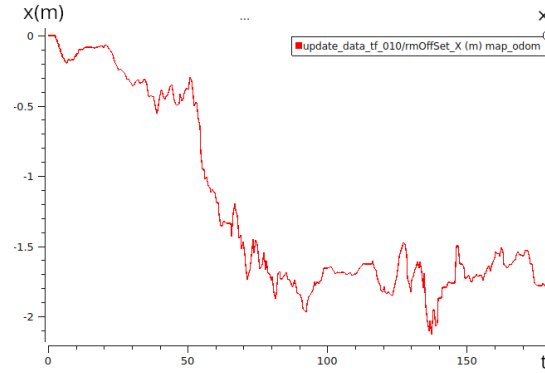
Correcting translation



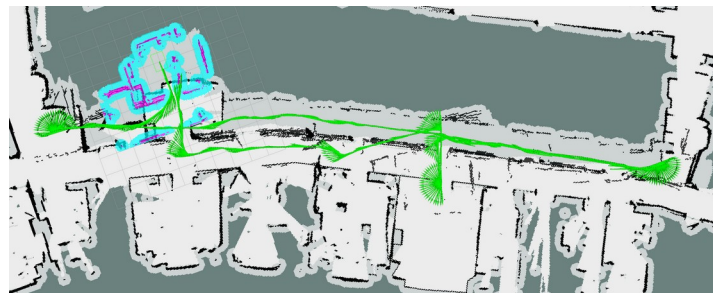
Results Mini correction with AMCL



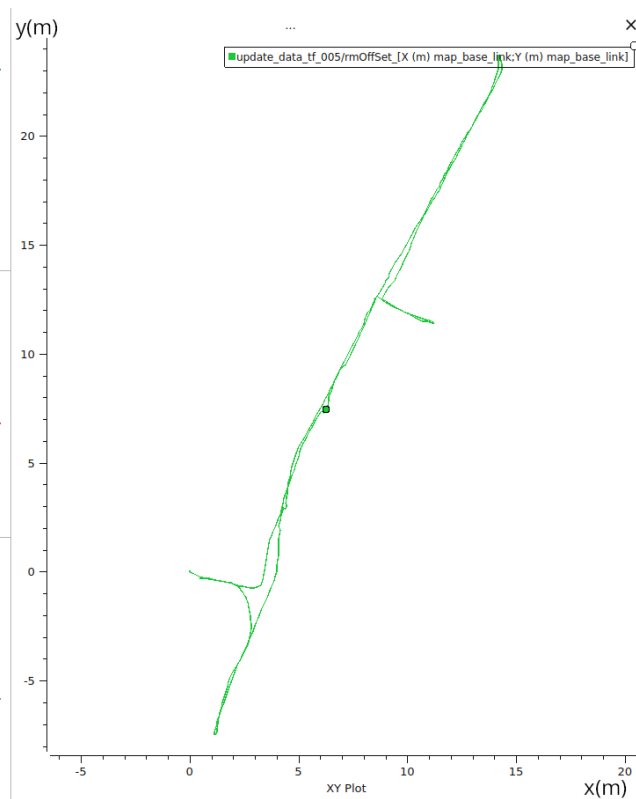
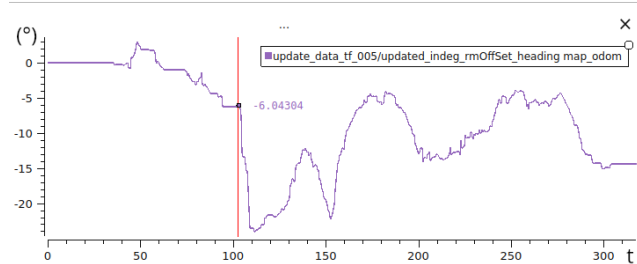
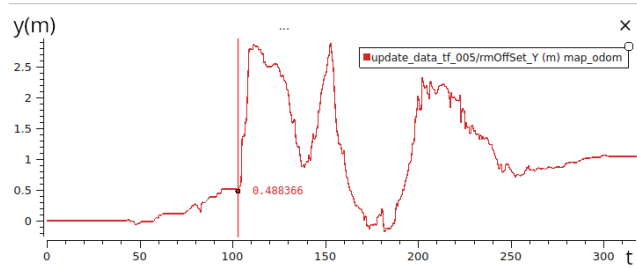
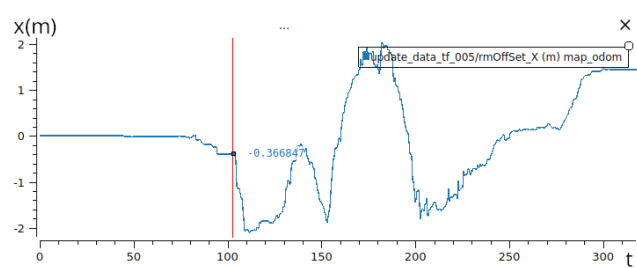
$\alpha 1$ (-)	$\alpha 2$ (-)	$\alpha 3$ (-)	$\alpha 4$ (-)	distance (m)	data name
0.0	0.0	0.0	0.0	1.92	-
0.001	0.0	0.0	0.0	1.32	000
0.01	0.0	0.0	0.0	0.60	001
0.1	0.0	0.0	0.0	0.64	002
0.0	0.001	0.0	0.0	0.58	003
0.0	0.1	0.0	0.0	0.51	004
0.0	0.0	0.001	0.0	2.01	005
0.0	0.0	0.01	0.0	2.27	006
0.0	0.0	0.1	0.0	2.32	007
0.0	0.0	0.0	0.1	3.08	008
0.01	0.01	0.01	0.0001	0.53	009
0.01	0.01	0.1	0.0001	0.69	010



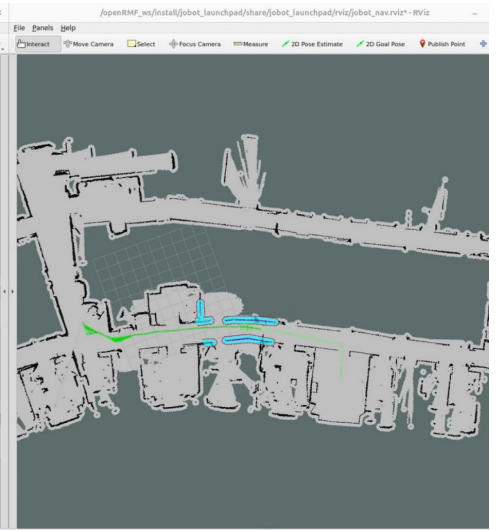
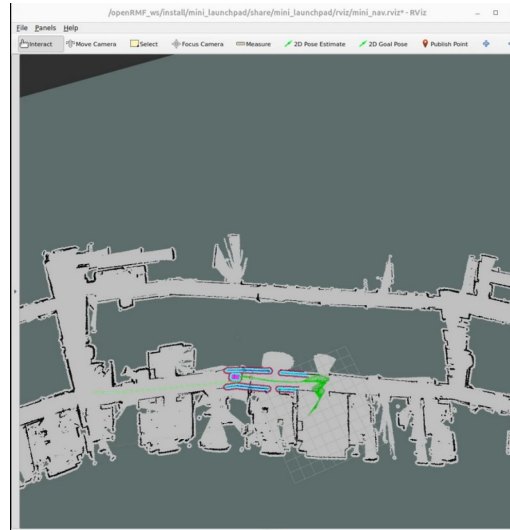
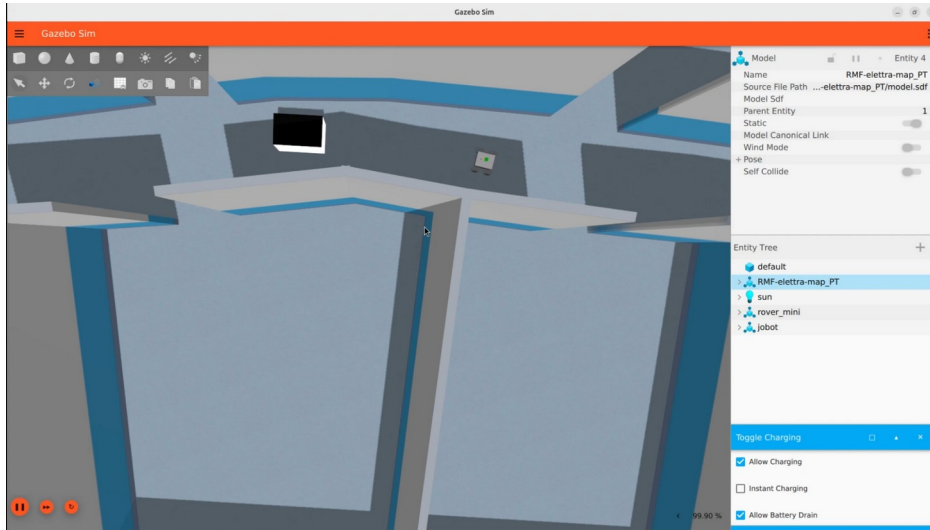
Results Jobot correction with AMCL



$\alpha1$ (-)	$\alpha2$ (-)	$\alpha3$ (-)	$\alpha4$ (-)	distance (m)	data name
0.0	0.0	0.0	0.0	1.63	-
0.001	0.0001	0.0001	0.0001	1.58	000
0.01	0.0001	0.0001	0.0001	2.16	001
0.01	0.001	0.0001	0.0001	0.71	002
0.01	0.001	0.001	0.0001	0.87	003
0.01	0.001	0.001	0.0001	0.74	004
0.01	0.001	0.001	0.001	0.59	005

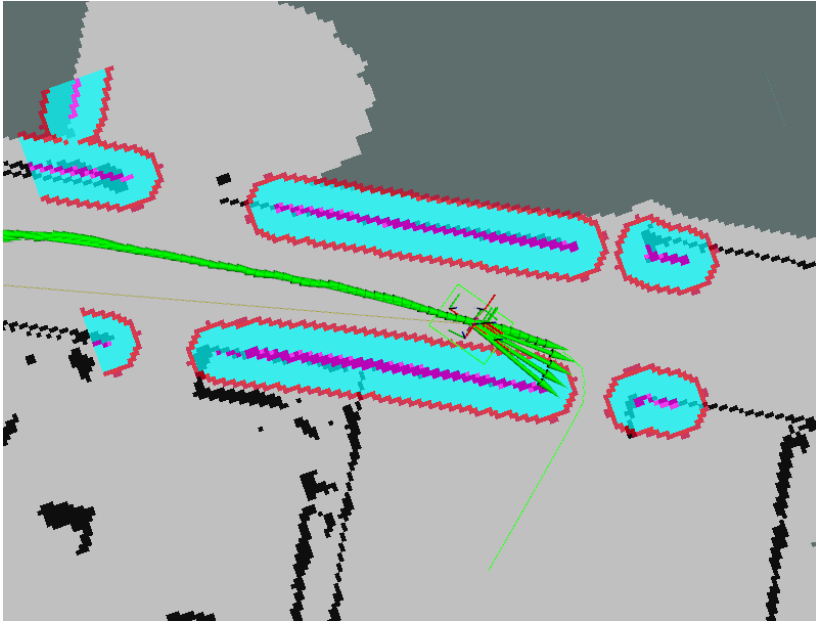


Results in simulation



Discovery in simulation

Error with the controller



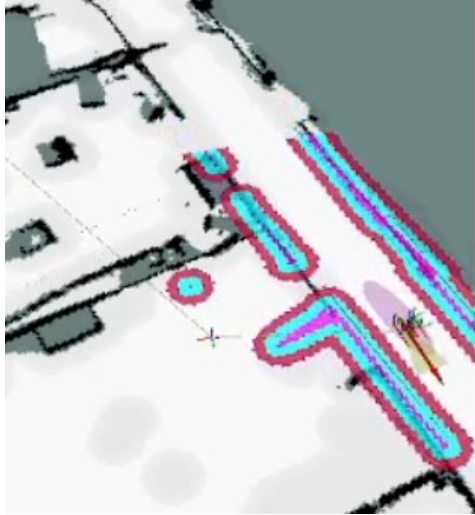
DWB Controller



MPPI Controller

Discoveries in reality

- Miss alignment from local costmap and global costmap



System fail to generate a path to enter the room



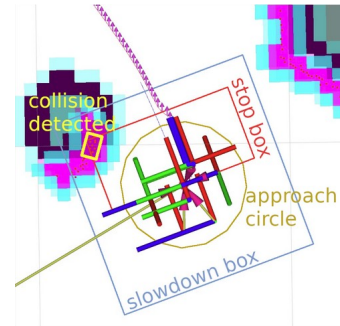
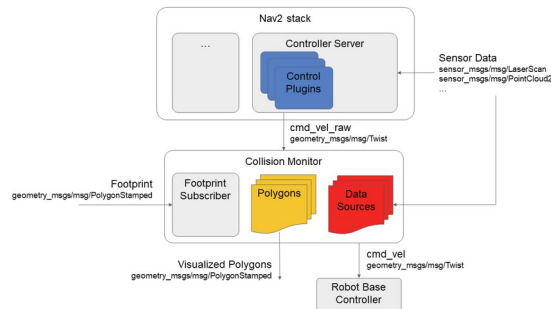
System attempts to turn left before recovering thanks to the BT recovery action

Discoveries in reality

- Obstacle detection not visible

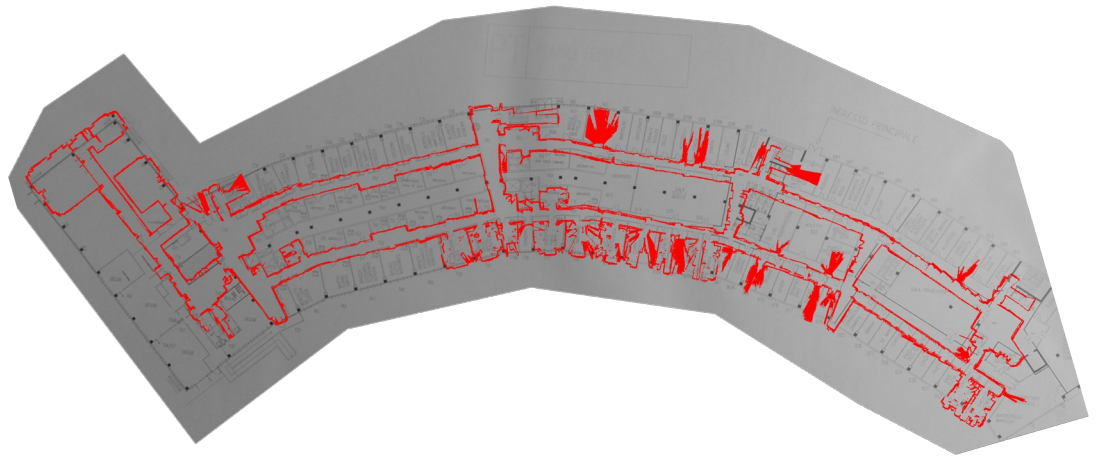


- Improve safety



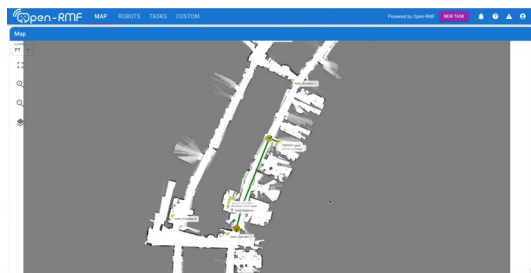
Discovery in reality

- Misalignment Cartographer and building map



- Communication
 - ROS_DOMAIN_ID
 - DDS properties
 - namespace

Build a system manager for the fleet Mini and for the fleet Jobot

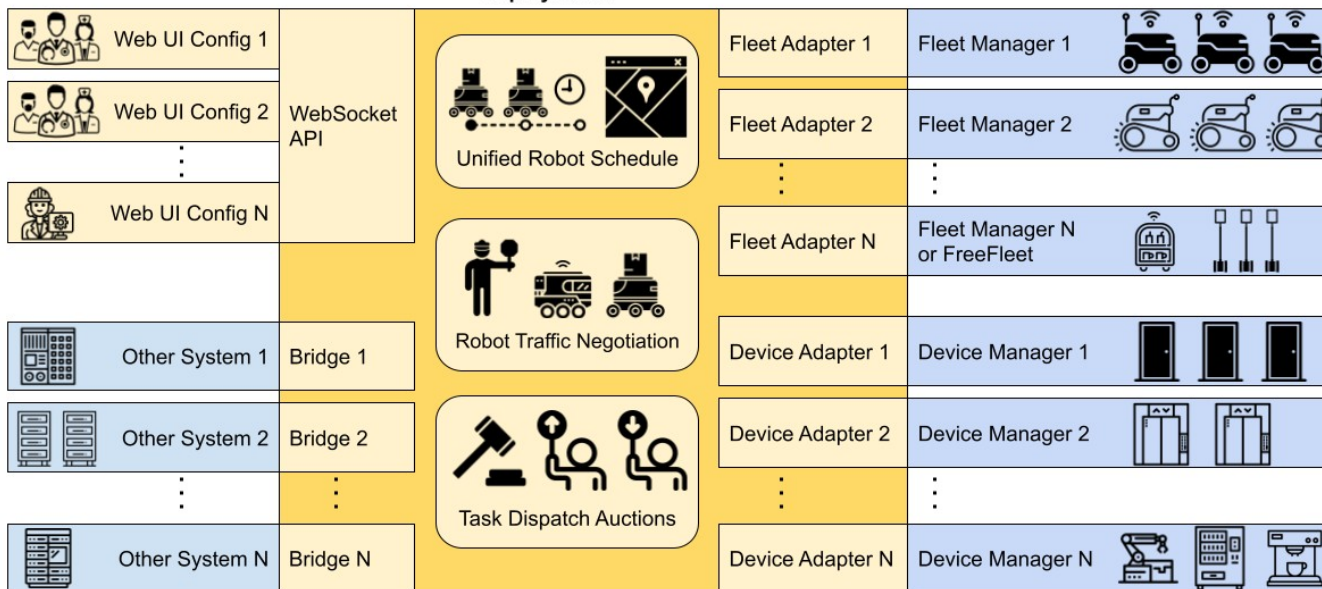


rmf_web

Deployment-specific configuration of Web UI's and bridges to other IT systems

RMF features common to all deployments

Deployment-specific collection of "RMF adapters" and vendor-provided "managers"

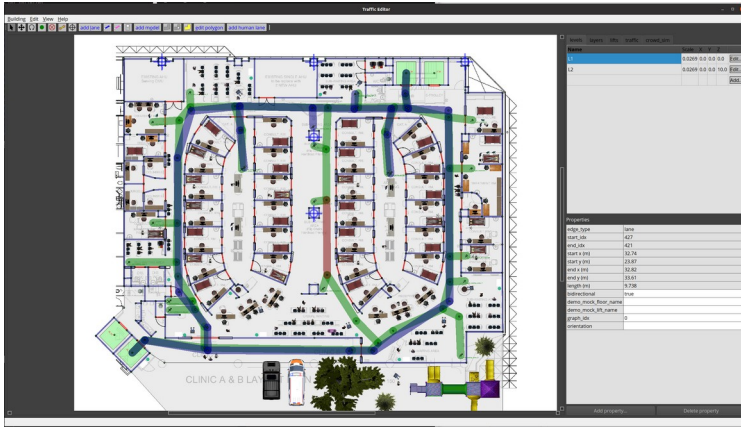


rmf_task_dispatcher

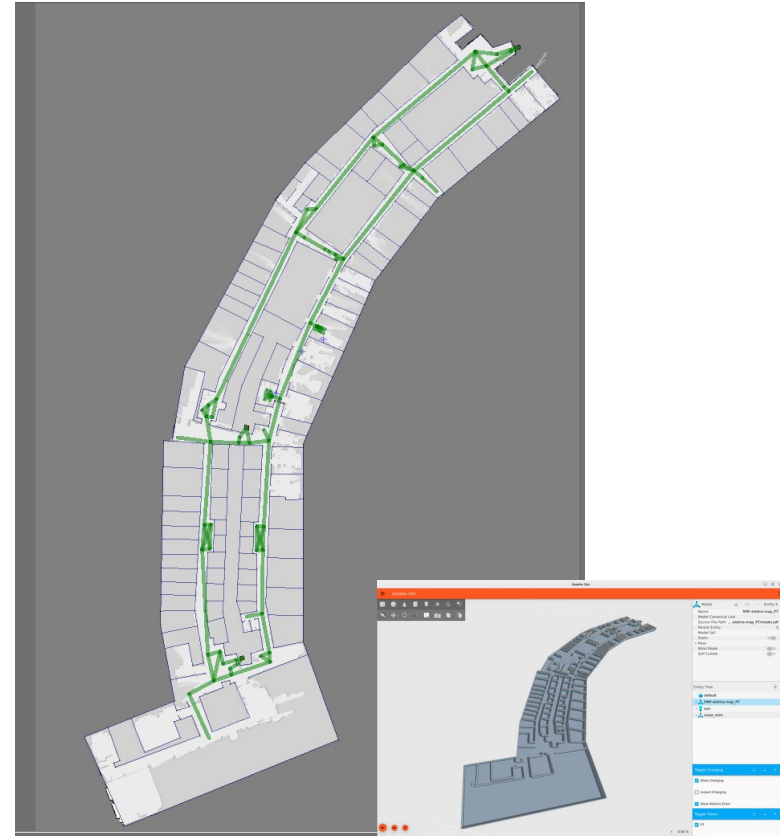


Open-RMF - Navigation graph

rmf_traffic_editor

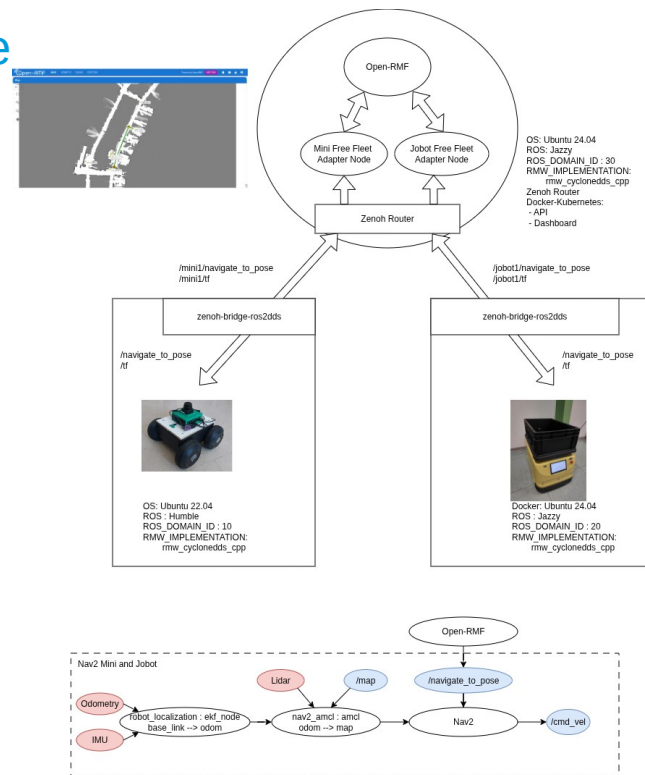
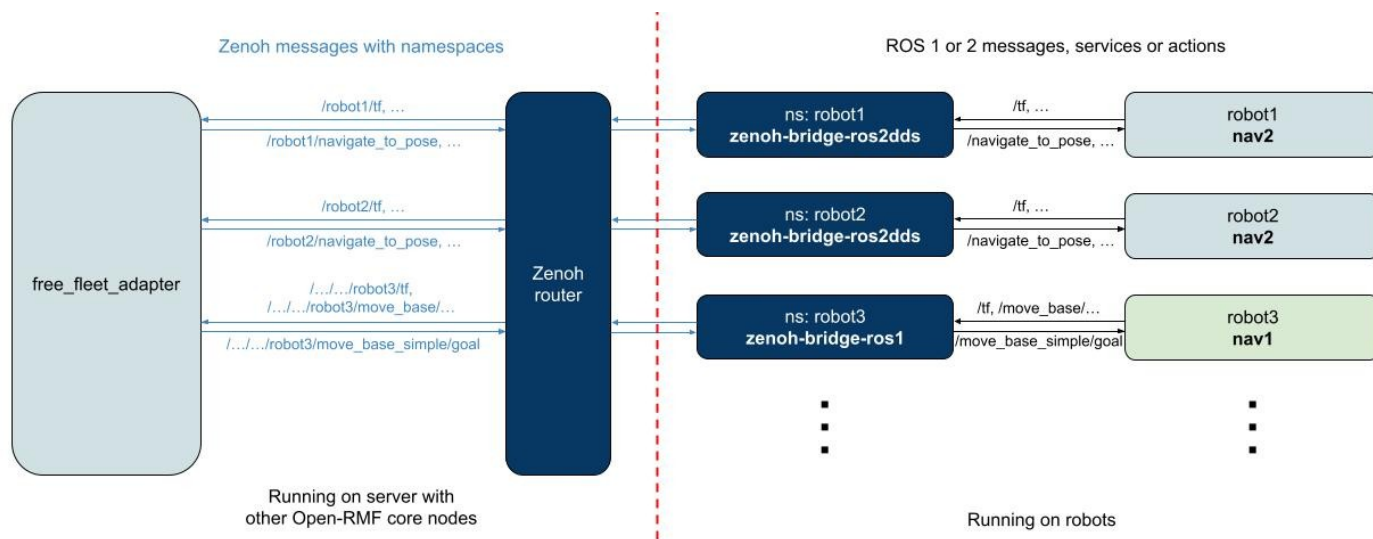


NEW!
rmf_site

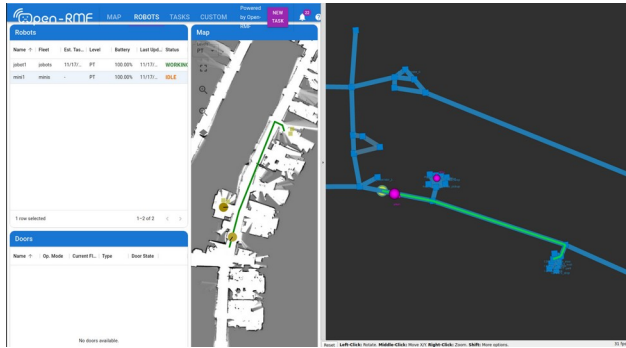


Free Fleet

a ROS 2 dedicated Fleet adapter based on fleet_adapter_template



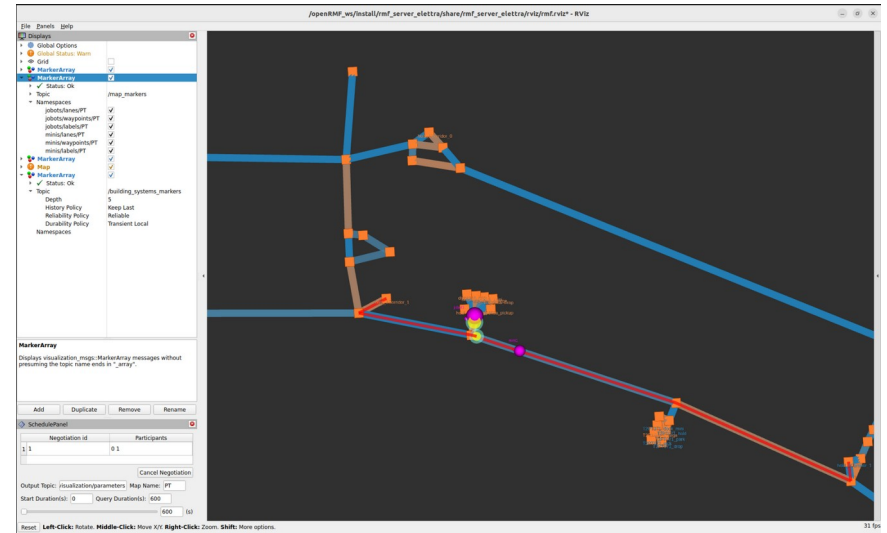
Open-RMF results



Patrol task without negotiation



Mini waiting the Jobot



Patrol task with negotiation

Conclusion

- ROS is an fast and easy solution to build an AMR
 - Modularity
 - Diffuse
 - Open-Source
- Open-RMF give all the tools for fleer management:
 - Give web interfaces
 - Give an easy template for adding a fleet
 - Solve the problems of congestion in problematic environments

Data and code

All the data is available at :

<http://dx.doi.org/10.1201/9780429489105>

While the code:

https://gitlab.elettra.eu/open-rmf-tesi/tesi_mag.git